Users's Manual for CPU Simulators
CLI and Lisp Programming

Brent Seidel
Phoenix, AZ

September 1, 2026

Note that this is a draft version and not the final version for publication.

Contents

Chapter 1

Introduction

This project is for a CLI interface to the CPU simulator project. In addition to using the simulations this way, it can also serve as an example for embedding a simulator in another application. For more information on the simulations themselves and their internals, see the simulations document.

1.1 Disclaimer

I have tried to make the documentation and software as accurate as possible. However, it is inevitable that errors exist. Should you find an error, feel free to write an issue on the GitHub project and I will look at it. For software errors especially, please provide information as to what change is needed. If you would like some particular hardware added, feel free to request that as well. Note that this is a hobby project maintained by one person who has other demands on their time.

1.2 License

Chapter 2

How To Obtain

This package is currently available on GitHub at `https://github.com/BrentSeidel/Sim-CPU`

2.1 Dependencies

2.1.1 Ada Libraries

The following Ada packages are used:

- `Ada.Calendar`
- `Ada.Characters.Latin_1`
- `Ada.Containers.Indefinite_Ordered_Maps`
- `Ada.Direct_IO`
- `Ada.Exceptions`
- `Ada.Integer_Text_IO`
- `Ada.Streams`
- `Ada.Strings.Fixed`
- `Ada.Strings.Maps.Constants`
- `Ada.Strings.Unbounded`
- `Ada.Text_IO`
- `Ada.Text_IO.Unbounded_IO`
- `Ada.Unchecked_Conversion`

2.1.2 Other Libraries

This library depends on the root package `BBS` available at `https://github.com/BrentSeidel/BBS-Ada` or using alire via "`alr get bbs`" and the `BBS.Lisp` package available at `https://github.com/BrentSeidel/Ada-Lisp` or using alire via "`alr get bbs_lisp`".

Chapter 3

Usage Instructions

3.1 Using Alire

Alire automatically handles dependencies. To use this in your project, just issue the command "`alr with bbs_simcpu`" in your project directory. To build the standalone CLI program, first obtain the cli using "`alr get simcpucli`". Change to the appropriate directory and use "`alr build`" and "`alr run`". To use the load CP/M program, use "`alr get loadcpm`". Change to the appropriate directory and use "`alr build`" and "`alr run`".

3.2 Using `gprbuild`

This is a library of routines intended to be used by some program. To use these in your program, edit your `*.gpr` file to include a line to `with` the path to `bbs_simcpu_noalr.gpr`. Then in your Ada code `with` in the package(s) you need and use the routines.

This is also available as a standalone program with a command line interface (CLI). This can be used for development or debugging. To build the CLI, just build the `simcli_noalr.gpr` project file, after making sure that the dependencies are in the expected places.

There is another program `loadcpm` available to create a bootable CP/M floppy disk image. It is build using the `loadcpm_noalr.gpr` project. It takes an Intel hex file image and writes it to a floppy disk image.

Chapter 4

General

4.1 Available Simulators

Several simulators are available for use. Each simulator may also have variation. So, one simulator may provide variations for different processors in a family of processors.

4.1.1 8080 Family

The 8080 simulator has variants defined for the 8080, 8085, and Z-80 processors. These are 8 bit processors with an 8 bit data bus and a 16 bit address bus. In addition to a memory bus, these processors also include an I/O bus with 8 bit I/O port addressing. See [12] for more information about the 8080/8085. See [17] and [16] for more information about the Z-80.

Currently, the 8080 family does not have memory mapped I/O. This may be added at some time in the future.

When loading data using the `load` routine, the specified file is assumed to be in Intel Hex format.

4.1.2 68000 Family

The 68000 simulator has variants defined for the 68000, 68008, 68010, and CPU32. Only the 68000 and 68008 are currently implemented. The 68010 and CPU32 are for future development. Internally, these are 32 bit processors with 32 bit data and 32 bit address busses. These processors do not have a separate I/O address space. All I/O is mapped as part of the main memory. The external address and data bus sizes depend on the variant selected. See [13] and [14] for more information about the 68000 family.

The interrupt code is interpreted with the low order bits (7-0) representing the vector number and the next 8 bits (15-8) representing the priority. Interrupts with vectors that match internally defined exception vectors are ignored. Thus only interrupt vectors in the range 25-31 and 64-255 are processed.

When loading data using the `load` routine, the specified file is assumed to be in Motorola S-Record format.

4.1.3 6502 Family

The 6502 simulator currently has only a working variants defined for the 6502 processor. This is an 8 bit processors with an 8 bit data bus and a 16 bit address bus. This processor does not have a separate I/O address space. All I/O is mapped as part of the main memory. Three interrupts are define for normal interrupts, non-maskable interrupts, and reset. See [15] for more information about the 6502.

When loading data using the `load` routine, the specified file is assumed to be in Intel Hex format.

4.1.4 PDP-11 Family

Most of the 16 bit addressing members are supported, except the LSI-11, PDP-11/03. Future models with memory management are planned at some point. This is a line of 16 bit minicomputers that was developed in the 1970s and was popular through the 1990s. Models with no memory management had 16 address bit. With memory management, depending on the model, addresses of 18 or 22 bits was supported. These processors do not have a separate I/O address space. All I/O is mapped as part of the main memory. By convention, all I/O is located in the upper 8kBytes of memory. See [10] and [11] for more information about the PDP-11.

A wealth of information about the PDP-11 (and other DEC hardware) is on bitsavers at `https://www.bitsavers.org/pdf/dec/`. Software can also be found on bitsavers at `https://www.bitsavers.org/bits/DEC/`. Various diagnostic software can be found using the PDP-11 Diagnostics - Database at `https://www.retrocmp.com/tools/pdp-11-diagnostic-database/202-pdp-11-diagnostics-database`.

4.2 I/O Devices

A number of simulated I/O devices are provided. Some are designed to mimic real I/O devices while others are just for the simulator.

4.2.1 Clock

The main function of the clock device is to provide a periodic interrupt. It can be enabled or disabled and the time between interrupts can be set in $\frac{1}{10}$th of a second. The base time can be changed by editing the code. Note that the timing is only approximate and depends on many things. Also since the simulation is not running at the same speed as actual hardware the number of instructions between each interrupt will be different from actual hardware.

4.2.2 Serial Ports

Serial ports provide a way to provide a terminal interface to a simulator. Unlike real serial ports, output buffering is done by the host operating system and it is not obvious to the I/O device if the output has been completed or not. As a result, currently no status is set or interrupts generated on output. The output is assumed to complete instantly.

Basic Serial Port

The basic serial port was developed in early testing to send output to and get input from the terminal controlling the simulator. It worked well enough to test some initial concepts, but has been superseded by the single line telnet port.

Single Line Telnet Port

The single line telnet port provides a replacement for the basic serial port. This device has an `init` routine that is used to set the TCP port. Using `telnet` to connect to this port connects to the device and will allow communication with software running on the simulator. If enabled, this device can provide interrupts to the simulator when characters are received.

Multi-Line Telnet Port

The multi-line telnet port combines 8 ports into a single simulated device. The `init` routine specifies the starting TCP port. It and the next seven ports can be accessed by `telnet` to communicate with software running on the simulator. This uses a single exception and requires fewer addresses than having 8 single line telnet ports.

Tape

From the point of view of the host program, the "paper" tape just reads bytes and writes bytes. The read bytes come from an attached file and the written bytes are written to an attached file (note that the should be two different files). One could also imagine this as a cassette tape interface if that better fits the computer you're simulating.

4.2.3 Disk Interfaces

The initial `disk_ctrl` device was designed with 8 inch floppy disks in mind for use by CP/M. It has been extended to allow other disk geometries, but follows the model of tracks, sectors per track, and heads for defining a disk. It currently supports 16 bit DMA, but may be extended to support 32 bit DMA. All data transfers occur between the request and the next instruction processed by the simulator.

Additional disk controllers based on actual hardware have been developed to support the PDP-11 CPU. Enough of the hardware has been simulated to support the RT-11 operating system. The controllers supported here are RK11 and RK611. The TM11 tape controller is also internally treated as a disk drive. This may change in the future.

4.3 Device Names

Each device has a name that can be used to refer to it. The name consists of an alphabetic device code followed by a decimal unit number. If no unit number is specified, 0 is assumed. A unit number of zero is special. It is used to refer to a generic device, such as when attaching a device - the unit number is assigned during the attachment. The currently defined device codes are:

CLK – A clock device to generate a periodic interrupt.

CON – A console serial device. In most cases, this should not be used.

FD – Main disk controller with a track and sector interface. It started out as a floppy disk controller, but geometries can be defined beyond that.

HD – An experimental disk controller using just block numbers. It is not yet implemented and tested.

MUX – An 8 channel terminal multiplexer using a telnet interface.

PRN – A simple printer output device.

PTP – A paper tape interface. It can read and write simulated tapes.

TEL – A single channel terminal device using a telnet interface.

The following devices simulate actual hardware for PDP-11 computers.

DL11 – A single line serial interface, see [6].

DZ11 – An 8 line serial interface multiplexer, see [9].

KW11 – A clock for the PDP-11 series of computer, see [1].

RK11 – A disk controller that can have up to 8 attached RK05 simulated drives, see [5].

RK611 – A disk controller than can have up to 8 attached RK07 simulated drives (RK06 drives are not supported), see [8].

PC11 – A combination paper tape reader/punch, see [3].

TM11 – A magnetic tape controller that can have up to 8 attached tape drives. Note that this is internally treated as a disk drive so use the disk command to control it, see [7].

Chapter 5

Command Line Interface

5.1 Commands

A number of commands are provided to allow the user to run and interact with software running on the simulator. The commands and their exact operation may change depending on feedback from usage. All commands may be abbreviated to the first 4 character. Some may be abbreviated further.

; – A comment. Everything following on the line is ignored, except if the rest of the line is being used as a file name.

ATTACH <device> <addr> <bus> [<dev-specific>] – Attaches a specific I/O device at the bus address. Depending on the device, device specific parameters may be required.

BREAK <addr> – Sets a breakpoint at the specified address.

CONTINUE – Clear a simulation halted condition. This can be abbreviated to `C`.

CPU <cpu-name> – Set the CPU to be simulated. This can only be done once per session and must be done before any command that requires a CPU.

DEP <addr> <byte> – Deposit a byte into memory.

DISK – Commands related to floppy disks and image files (see below for more details).

DUMP <addr> – Dump a block of memory starting at the specified address. This can be abbreviated to `D`.

EXIT – Exit the simulation (synonym for `QUIT`).

GO <addr> – Sets the execution address.

INTERRUPT – Can be abbreviated to `INT`. Used to enable or disable interrupt processing by the simulated CPU.

LISP [<filename>] – Starts the Tiny-Lisp environment. If a filename is specified, Lisp commands come from that file.

LIST [<**devname**>] – Lists all the I/O devices attached to the simulator. If <devname> is supplied, it provides information for that specific device.

LOAD <**filename**> – Loads memory with the specified file (typically Intel Hex or S-Record format).

TAPE – Commands relating to printer and attached file (see below for more details).

QUIT – Exit the simulation (synonym for `EXIT`).

REG – Display the registers and their contents.

RESET – Calls the simulator's `init` routine.

RUN – Runs the simulator. While running, it can be interrupted by a control-E character. This can be abbreviated to `R`.

SET – Sets certain simulator options.

STEP – Execute a single instruction, if the simulation is not halted.

TAPE – Commands relating to paper (or cassette) tape and attached files (see below for more details).

TRACE <**value**> – Sets the trace value. The value is interpreted by the simulator and typically causes certain information to be printed while processing.

UNBREAK <**addr**> – Clears a breakpoint at the specified address. For some simulators, the address is ignored and may not need to be specified. In this case, all or the only breakpoint is cleared.

5.1.1 Attach Command

The attach command is used to attach a device to the current CPU simulator. The device is specified as a generic device name. The address is given in decimal. The bus is:

IO – Use the I/O bus (only available on some CPUs).

MEM – Use memory mapped I/O (only available on some CPUs).

Some of the devices take additional parameters.

CLK – Takes one additional decimal value for the exception code to be presented to the CPU when an interrupt occurs.

DL11 – Takes two additional decimal parameters. The first is the TCP/IP port for the first terminal (the additional terminals are assigned sequential port numbers). The second is a 32 bit integer combination of two exception codes to be presented to the CPU when an interrupt occurs. The lower order 16 bits are the receive vector and the upper 16 bits are the transmit vector.

FD – Takes one additional decimal parameter for the number of attached drives in the range 0-15, though 0 drives is rather pointless.

HD – Not yet implemented for ATTACH (experimental - under development).

KW11 – Takes one additional decimal value for the exception code to be presented to the CPU when an interrupt occurs.

MUX – Takes two additional decimal parameters. The first is the TCP/IP port for the first terminal (the additional terminals are assigned sequential port numbers). The second is the exception code to be presented to the CPU when an interrupt occurs.

PC11 – Takes one additional parameter. It is a 32 bit integer combination of two exception codes to be presented to the CPU when an interrupt occurs. The lower order 16 bits are the receive vector and the upper 16 bits are the transmit vector.

PRN – Takes no additional parameters.

PTP – Takes no additional parameters.

RK11 – Takes one additional decimal value for the exception code to be presented to the CPU when an interrupt occurs.

RK611 – Takes one additional decimal value for the exception code to be presented to the CPU when an interrupt occurs.

TEL – Takes two additional decimal parameters. The first is the TCP/IP port for the first terminal (the additional terminals are assigned sequential port numbers). The second is the exception code to be presented to the CPU when an interrupt occurs.

TM11 – Takes one additional decimal value for the exception code to be presented to the CPU when an interrupt occurs.

5.1.2 CPU Command

This command is deprecated. SET CPU is the preferred command.

5.1.3 Disk Commands

A number of subcommands are available for interfacing with simulated floppy disks and image files. The DISK command takes the form:

- DISK <controller> <subcommand> <parameters for subcommand>

The controller is the device name of an attached disk controller. The subcommands are:

CLOSE <drive number> – Closes the image file associated with the specified drive number.

OPEN <drive number> <file name> – Attaches the specified file to the specified drive.

READONLY <drive number> – Sets the specified drive to read-only.

READWRITE <drive number> – Sets the specified drive to read-write.

5.1.4 Interrupt Commands

A number of subcommands are available for interfacing with interrupts.

ON – Enables interrupt processing by the simulator (if supported).

OFF – Disables interrupt processing by the simulator. This may be useful for single-stepping through some code without getting interrupted by the clock or other devices.

SEND <interrupt number> – Sends the specified interrupt number to the simulator (if supported).

5.1.5 Set Commands

This allows setting of some simulator options. The current options are:

CPU – The same as the CPU command above. The following CPUs are currently defined:

8080 – The Intel 8080 processor. An 8 bit processor that can address up to 64kBytes of memory and 256 I/O ports. See [12].

8085 – Made by Intel. Very similar to the 8080 with mostly hardware enhancements, with 2 added instructions. See [12].

Z80 – Made by Zilog. It is mostly upward compatible with the 8080 (one flag is different on some instructions). The added instruction codes for the 8085 are completely different instructions on the Z80. See [17] and [16].

68000 – The Motorola 68000 processor. This is internally a 32 bit processor with a 16 bit data bus, and 24 address lines. It can address up to 16MBytes of memory. See [13] and [14].

68008 – A stripped down version of the 68000. The core is the same, but it has an 8 bit address bus and only 20 address lines. See [13] and [14].

6502 – A simple 8 bit processor with 16 bits of addressing. See [15].

PDP-11/04 – A version of the PDP-11 series of minicomputers. See [4] and [10].

PDP-11/05 – A version of the PDP-11 series of minicomputers. See [2], [4] and [10].

PDP-11/10 – The same as the PDP-11/05.

PDP-11/15 – A version of the PDP-11 series of minicomputers.

PDP-11/20 – The same as the PDP-11/15.

PDP-11/34 – Under development.

PDP-11/35 – Under development. See [4].

PDP-11/40 – The same as PDP-11/35.

It is likely that more PDP-11 models will be added.

PAUSE – Sets either the pause count or the pause character. This controls how often to check for the pause/interrupt character while the simulator is running. Since the check for the character takes much longer than simulating an instruction, increasing this value can speed the simulation.

5.1.6 Tape Commands

The tape interface consists of a reader called `RDR` and a writer called `PUN` (for punch). Files can be opened or closed for either of these devices. The TAPE command takes the form:

- TAPE <controller> <subcommand> <parameters for subcommand>

The controller is the device name of an attached tape controller. The subcommands are:

CLOSE <PUN or RDR> – Closes the file attached to the reader or punch.

OPEN <PUN or RDR> <file name> – Attaches the specified file to the reader or punch.

5.1.7 Printer Commands

The printer device simply writes data sent to it to an output file. If no file is open, the data is discarded. The PRINT command takes the form:

- PRINT <controller> <subcommand> <parameters for subcommand>

The controller is the device name of an attached printer controller. The subcommands are:

CLOSE – Closes the file attached to the printer.

OPEN <file name> – Attaches the specified file to the printer.

5.2 Lisp Programming

The Lisp environment is based on Ada-Lisp, available at `https://github.com/BrentSeidel/Ada-Lisp`, with a few commands added to interface with a simulator. Refer to the Ada-Lisp documentation for details about the core language. This section just describes the additional commands.

5.2.1 Attach

Inputs

The following inputs are expected:

- A string representing the device name.
- An integer for the device address.
- A string representing the address bus used ("IO" or "MEM").
- Additional items specific to the device may be required.

Outputs

None

Description

Attaches an I/O device at the specified address and address bus. Refer to the Attach command (section 5.1.1) for more information.

5.2.2 Disk-Close

The following inputs are expected:

- A string representing the disk controller name.
- An integer for the drive number.

Outputs

None

Description

Closes the file attached to the specified disk drive.

5.2.3 Disk-Geom

The following inputs are expected:

- A string representing the disk controller name.
- An integer for the drive number.
- A string representing the geometry (currently "IBM" for 8 inch diskettes or "HD" for a 5MB hard disk). It is planned to extend this to allow a list specifying arbitrary geometry as well.

Outputs

None

Description

Sets the disk geometry (tracks, sectors, heads) of the specified disk drive.

5.2.4 Disk-Open

The following inputs are expected:

- A string representing the disk controller name.
- An integer for the drive number.
- A string representing the file name to attach to the disk drive.

Outputs

None

Description

Opens a file and attaches it to the specified disk drive.

5.2.5 Go

Inputs

A number representing the starting address for program execution.

Outputs

None

Description

Sets the starting address for program execution. The next `(sim-step)` command executes the instruction at this location.

5.2.6 Halted

Inputs

An optional boolean that if set true, clears the halted state of the simulator. If set false, does nothing.

Outputs

If no input is provided, it returns a boolean representing the halted state,

Description

This is used to clear to test the halted state of a simulator.

5.2.7 Int-state

Inputs

None

Outputs

A simulator dependent integer representing the state of interrupts.

Description

Returns a simulator dependent integer representing the state of the interrupts. Typically it is something like 0 for interrupts disabled and 1 for interrupts enabled. For systems with priorities, it may be the processor priority.

5.2.8 Last-out-addr

Inputs

None

Outputs

The address used by the last output instruction.

Description

Returns the address used by the last output instruction. This is only meaningful if the processor being simulated has a separate output bus rather than memory mapped I/O.

5.2.9 Last-out-data

Inputs

None

Outputs

The data used by the last output instruction.

Description

The data used by the last output instruction. This is only meaningful if the processor being simulated has a separate output bus rather than memory mapped I/O.

5.2.10 Memb

Inputs

A memory address and an optional byte value.

Outputs

If no byte value is provided, it returns the byte at the memory address.

Description

Reads or writes a byte value in memory.

5.2.11 Meml

Inputs

A memory address and an optional long value.

Outputs

If no long value is provided, it returns the long at the memory address.

Description

Reads or writes a long value in memory.

5.2.12 Memw

Inputs

A memory address and an optional word value.

Outputs

If no word value is provided, it returns the word at the memory address.

Description

Reads or writes a word value in memory.

5.2.13 Num-reg

Inputs

None

Outputs

The number of registers defined by the simulator.

Description

Returns the number of registers defined by the simulator. This could be used for iterating through the registers and displaying their values.

5.2.14 Override-in

Inputs

Two integers representing the address of an I/O port and the data to provide.

Outputs

None

Description

Provides an address and data to override the next input instruction. If the instruction reads from the address, the provided data is read by the simulator instead of reading from the actual I/O device. Inputs from a different address work normally and any input instruction clears the override. This is intended for testing purposes.

5.2.15 Print-Close

The following inputs are expected:

- A string representing the printer controller name.

Outputs

None

Description

Closes the file attached to the specified printer.

5.2.16 Print-Open

The following inputs are expected:

- A string representing the printer controller name.
- A string representing the file name to attach to the printer.

Outputs

None

Description

Opens a file and attaches it to the specified printer.

5.2.17 Reg-val

Inputs

A register number

Outputs

The value of the specified register, or an error if the register number is not in range.

Description

Returns the value of the specified register.

5.2.18 Send-int

Inputs

An integer interrupt code.

Outputs

None.

Description

Sends the specified interrupt code to the simulator for processing. The processing done depends on the specific simulator. It may be ignored, be a vector number, be an instruction, or something else.

5.2.19 Set-pause-char

Inputs

An integer in the range 0-255 as the ASCII value of the pause character. Note that if you're on a system that doesn't use ASCII, then this is whatever the numerical value for the character is.

Outputs

The current pause character.

Description

Sets the pause character. Pressing this character while a simulation is running will cause the simulation to return to the command line. If no character is specified, it just returns the current pause character.

5.2.20 Set-pause-count

Inputs

A positive integer.

Outputs

The current value of the pause count.

Description

Sets the number of instructions to execute between checks for the pause character.

5.2.21 Sim-CPU

Inputs

A string containing the CPU name. See the "SET CPU" (section 5.1.5) command for valid CPU names.

Outputs

None

Description

Selects the CPU to simulate. This can only be done once and must be done before any command that attempts to access the CPU.

5.2.22 Sim-init

Inputs

None.

Outputs

None.

Description

Calls the initialization routine to the current simulator. The action is simulator dependent, but typically clears the registers. The memory may, or may not be cleared.

5.2.23 Sim-load

Inputs

A string containing a file name.

Outputs

None.

Description

Calls the load routine for the current simulator and passes the string containing the file name. The effect is simulator dependent, but the 8080 simulator will attempt to load the file as an Intel hex format file and the 68000 simulator will attempt to load the file as a Motorola S-record file. The PDP-11 simulator will load a binary file in absolute loader format.

5.2.24 Sim-step

Inputs

None.

Outputs

None.

Description

Executes one instruction or step of the simulator.

5.2.25 Tape-Close

The following inputs are expected:

- A string representing the tape controller name.
- A string representing the output ("PUN" for punch) or input ("RDR" for reader).

Outputs

None

Description

Closes the file attached to the specified tape drive unit.

5.2.26 Tape-Open

The following inputs are expected:

- A string representing the tape controller name.
- A string representing the output ("PUN" for punch) or input ("RDR" for reader).
- A string representing the file name to attach to the tape drive.

Outputs

None

Description

Opens a file and attaches it to the specified tape drive unit.

5.2.27 Examples

The Lisp environment is useful for writing automated tests and for writing utilities to enhance the CLI.

Watch a Memory Location

The following Lisp program will execute instructions until the specified memory location changes.

```
(defun watch (addr)
  (setq old-value (meml addr))
  (dowhile (= old-value (meml addr))
    (sim-step)))
```

Bibliography

[1] Digital. *KW11-L Line Time Clock Manual*. Digital Equipment Corporation, 1973.

[2] Digital. *PDP-11/05 Computer Manual*. Digital Equipment Corporation, 1973.

[3] Digital. *PC11 High-Speed Reader/Punch and Control Manual*. Digital Equipment Corporation, 1974.

[4] Digital. *PDP11 04/05/10/35/40/45 Processor Handbook*. Digital Equipment Corporation, 1975.

[5] Digital. *RK11-D and RK11-E Moving Head Disk Drive Controller Manual*. Digital Equipment Corporation, 1975.

[6] Digital. *DL11 Asynchronous Line Interface User's Manual*. Digital Equipment Corporation, 1976.

[7] Digital. *TMB11/TS03 DECmagtape System User's Manual (TMB11-M System)*. Digital Equipment Corporation, 1976.

[8] Digital. *RK06/RK07 Disk Drive User's Manual*. Digital Equipment Corporation, 1978.

[9] Digital. *DZ11 User's Guide*. Digital Equipment Corporation, 1979.

[10] Digital. *PDP11 04/34a/44/60/70 Processor Handbook*. Digital Equipment Corporation, 1981.

[11] Digital. *PDP-11 Architecture Handbook*. Digital Equipment Corporation, 1984.

[12] Intel. *8080/8085 Assembly Language Programming*. Intel, 1978.

[13] Motorola. *M68000 16/32-Bit Microprocessor Programmer's Reference Manual*. Prentice-Hall, 4th edition, 1984.

[14] Motorola/NXP. *Motoroa M68000 Programmer's Reference Manual*. Motorola/NXP, 1992.

[15] MOS Technology. *MSC6502 Microcomputer Family Programming Manual*. MOS Technology, July 1976.

[16] Sean Young and Jan Wilmans. The undocumented z80 documented, 2005.

[17] Zilog/IXYS. *Z80 CPU User Manual*. Zilog, 2016.